Инструкция по интеграции готовых скриптов с Web UI («Сверх.Рой») — v2

Положите файл ui_bridge_node-v2.py (переименовав в ui_bridge_node.py) рядом с вашими скриптами и импортируйте класс UIConnector в свои существующие ROS 2 ноды. Карта на сайте содержит только разрешенные клетки (1 — можно летать) и запретные зоны (-1 — нельзя летать).

1. Для автора скрипта полёта по ArUco-карте

Вставьте 2 строчки в вашу существующую ROS 2 ноду навигации, чтобы передавать координаты (x, y от 0 до 4) и напряжение АКБ на сайт:

```
from ui_bridge_node import UIConnector

# В __init__ вашей ноды:
self.ui = UIConnector(self)

# В функции, где вы получаете текущую метку ArUco (x, y) и заряд батареи:
self.ui.send_aruco_and_battery(drone_id=1, x=current_x, y=current_y, battery=11.8)
```

2. Для автора скрипта распознавания цветов (OpenCV)

Вставьте вызов send_color_detection в место, где ваш скрипт находит цветной квадрат. Сайт сам посчитает сумму без дублей и выведет картинку:

```
from ui_bridge_node import UIConnector

# В __init__ вашей ноды зрения:
self.ui = UIConnector(self)

# Когда квадрат распознан (color: "red", "yellow", "blue" или "green" в симуляторе):
self.ui.send_color_detection(
   drone_id=1, x=aruco_x, y=aruco_y, z=1.5,
   color=detected_color, confidence=0.95,
   message="Найден квадрат", cv_image=frame
)
```

3. Для автора скрипта парной работы (управление миссией)

Привяжите функции старта и аварийной остановки к кнопкам сайта (либо читайте скачанный файл matrix.json, где 1 — можно летать, -1 — нельзя летать):

```
from ui_bridge_node import UIConnector

# В __init__ вашей ноды роя:
self.ui = UIConnector(self, on_start_callback=self.start_swarm,
on_kill_callback=self.emergency_stop)

# В start_swarm(self, matrix) приходит матрица 5x5 из 1 (можно летать) и -1 (No-Fly).
```